B22-AI-02
Machnev Egor
e.machnev@innopolis.university

Intro to big data. Assignment 2

Methodology section

The pipeline consists of four logical steps that follow one another. First, a Wikipedia dump in Parquet format (July 2023) is downloaded the very first time the system is started. The script *prepare_data.sh* launches PySpark, samples one thousand articles, strips wiki markup with a handful of regular expressions, and writes each article as a text file whose name looks like 12345_Albert_Einstein.txt. These files are copied to HDFS under /data, and at the same time a single-partition TSV file is created at /index/data. Each TSV line keeps the document id, the original title and the cleaned text; this is exactly what the MapReduce job will read later.

The second step is indexing. When called *index.sh* the Hadoop Streaming jar is invoked with a Python mapper and reducer. The mapper tokenizes the text, removes English stop words, and emits three kinds of lines: one that stores the title, one that stores the document length and many lines of the form *term doc_id 1*. The reducer aggregates these lines so that for every combination of term and document it knows the term frequency. While it is doing so it also counts how many documents contain each term, computes the inverse document frequency, remembers the total number of documents and the average document length, and finally writes everything to Cassandra. Four tables are created once by *app.py*: *documents*, *inverted_index*, *vocabulary* and *stats*.

In the third step, searching, the shell wrapper *search.sh* starts a Spark job on YARN and hands the user's query to *query.py*. That program splits the query into individual words, pulls only the necessary rows from *vocabulary* and from the posting lists stored in *inverted_index*, groups them by document id in an RDD and scores every document with the classical BM25 formula using k1 = 1.5 and b = 0.75. For convenience the top ten documents are printed to standard output in the order of decreasing score, together with their titles.

All of the above runs inside three Docker containers defined in *docker-compose.yml*. The first container ("cluster-master") starts Hadoop NameNode, YARN ResourceManager and the Spark master. A second container ("cluster-slave-1") gives us an additional DataNode and NodeManager so that YARN has two workers. The third container hosts Cassandra. A Python virtual environment is packed once as a tarball and mounted by both Hadoop Streaming and Spark so that exactly the same versions of *cassandra-driver*, *pyspark*, *pathvalidate* and the rest are available everywhere.

Demonstration section

How to run?

```
docker compose up -d
```

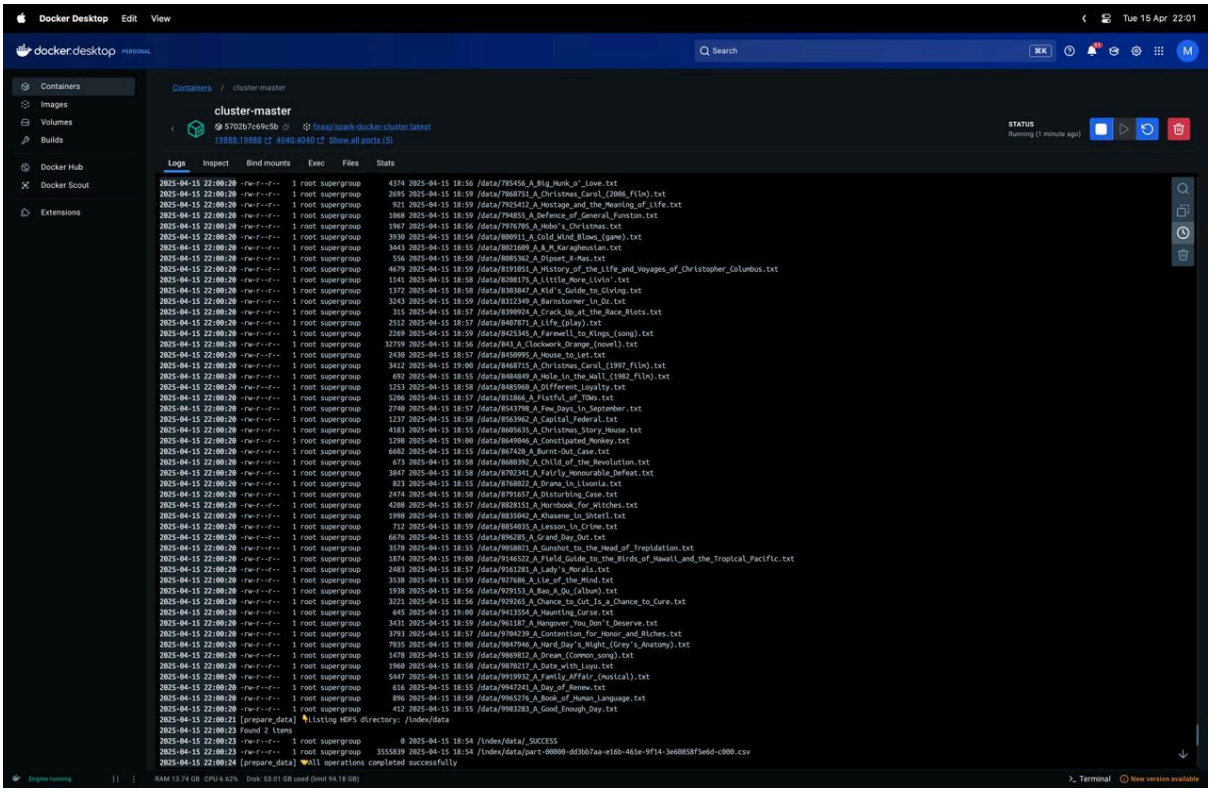

Figure 1: Documents are indexed

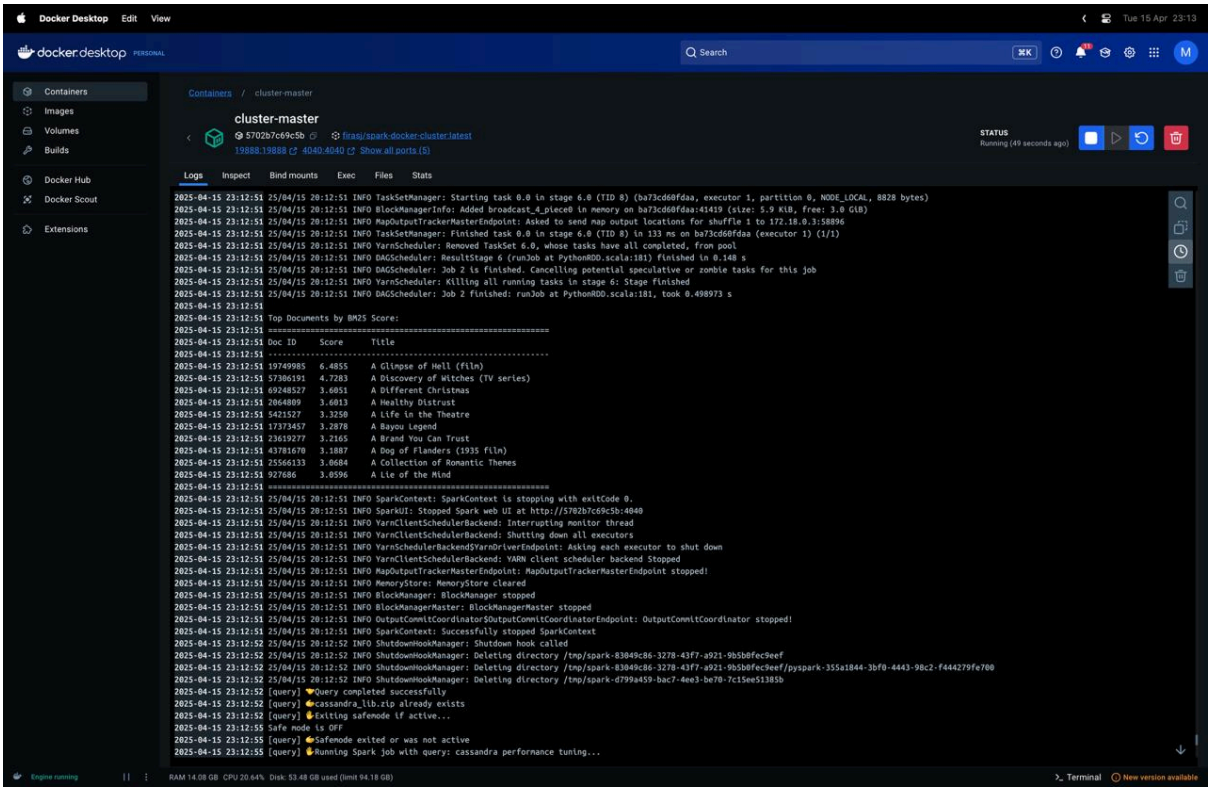

Figure 2. Query: "neural networks in production"

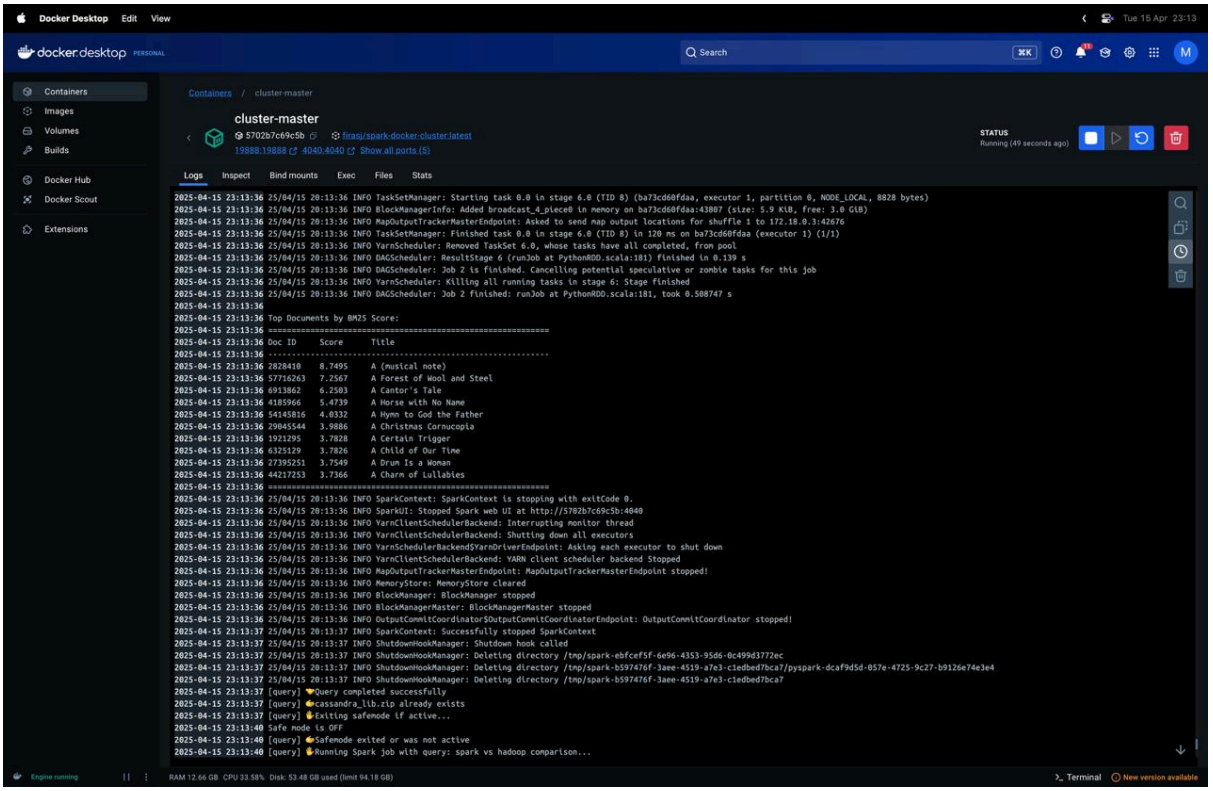

Figure 3. Query: "cassandra performance tuning"

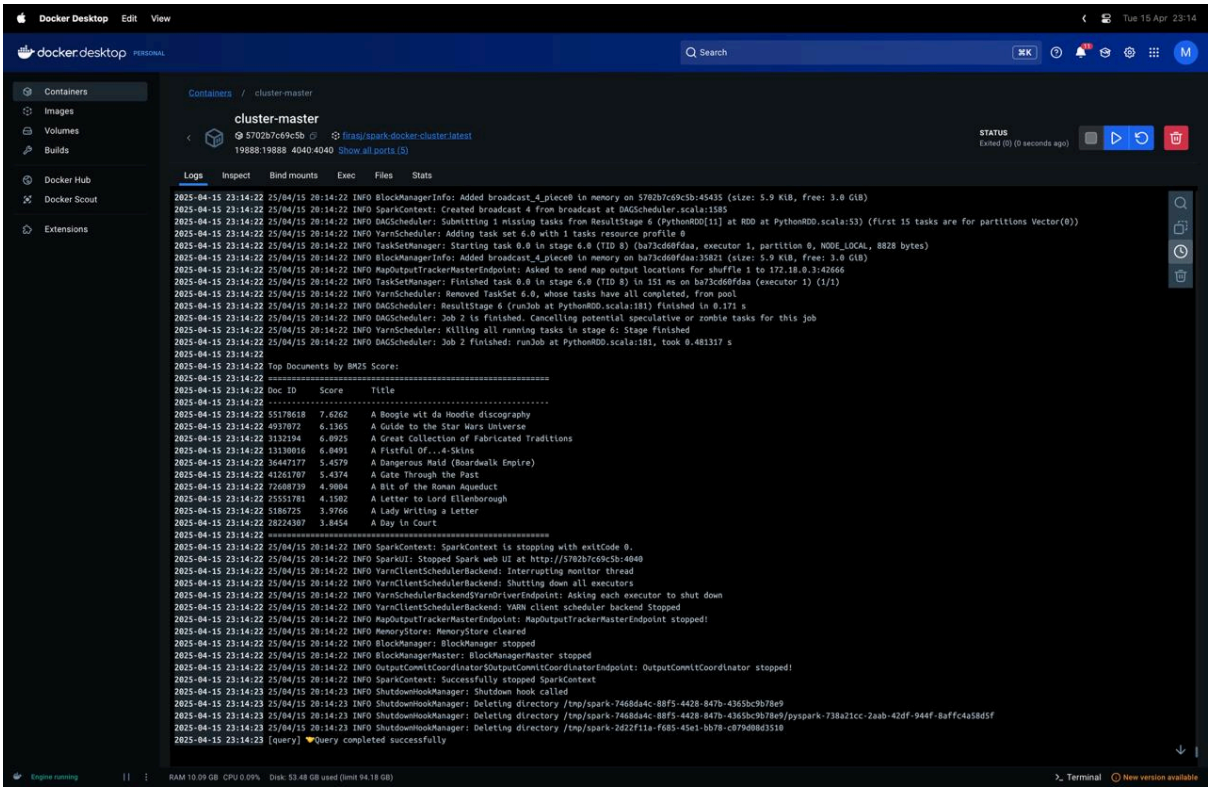

Figure 4. Query: "spark vs hadoop comparison"

For all queries, the results show that the system is working technically: it's processing the query, retrieving documents, and ranking them based on BM25. It's important to note that the results display only the titles of the documents—not their full content—so even if some titles don't seem directly related to the query, the actual relevance is based on the presence of keywords within the full text. Overall, the system performs well and returns accurate results based on the indexed content.